

Nome, MATRICOLA

Considerando il processore MIPS64 e l'architettura descritta in seguito:

- Integer ALU: 1 clock cycle
- Data memory: 1 clock cycle
- FP multiplier unit: pipelined 7 stages
- FP arithmetic unit: pipelined 2 stages
- FP divider unit: not pipelined unit that requires 7 clock cycles
- branch delay slot: 1 clock cycle, and the branch delay slot disabled
- forwarding enabled
- it is possible to complete instruction EXE stage in an out-of-order fashion.

Usando il frammento di codice riportato, si calcoli il tempo di esecuzione dell'intero programma in colpi di clock e si completi la seguente tabella.

```

;   for (i = 0; i < 100; i++) {
;       v4[i] = (v1[i]/(v2[i]*v3[i]));
; }

```

[illegible]



Nome, MATRICOLA

Domanda 2

Considerando il programma precedente, quali sono le copie di istruzioni che beneficiano principalmente dell'architettura Harvard del processore e perché? motivare la risposta.

- L.D F2,V3(R1) – LD F1,V1(R1)
- S.D F4,V4(R1) – BNEZ R2,LOOP

Queste istruzioni accedono a memoria di dati e di codice contemporaneamente; questo è possibile grazie all'utilizzo dell'architettura Harvard.

Nome, MATRICOLA

Domanda 3

Considerando il programma precedente e l'architettura del processore superscalare descritto in seguito; completare la tabella relativa alle prime 3 iterazioni.

Processor architecture:

- Issue 2 instructions per clock cycle
- jump instructions require 1 issue
- handle 2 instructions commit per clock cycle
- timing facts for the following separate functional units:
 - 1 Memory address 1 clock cycle
 - 1 Integer ALU 1 clock cycle
 - 1 Jump unit 1 clock cycle
 - 1 FP multiplier unit, which is pipelined: 6 stages
 - 1 FP divider unit, which is not pipelined: 8 clock cycles
 - 1 FP Arithmetic unit, which is pipelined: 2 stages
- Branch prediction is always correct
- There are no cache misses
- There are 2 CDB (Common Data Bus).

# iteration		Issue	EXE	MEM	CDB x2	COMMIT x2
1	l.d f2,v3(r1)	1	2m	3	4	5
1	l.d f3,v4(r1)	1	3m	4	5	6
1	mul.d f4,f2,f3	2	6x		12	13
1	l.d f1,v1(r1)	2	4m	5	6	13
1	div.d f4,f1,f4	3	13d		21	22
1	s.d f4,v4(r1)	3	5m			22
1	daddui r1,r1,8	4	5i		6	23
1	daddi r2,r2,-1	4	6i		7	23
1	bnez r2,loop	5	8j			24
2	l.d f2,v3(r1)	6	8m	9	10	24
2	l.d f3,v4(r1)	6	9m	10	11	25
2	mul.d f4,f2,f3	7	12x		18	25
2	l.d f1,v1(r1)	7	10m	11	12	26
2	div.d f4,f1,f4	8	21d		29	30
2	s.d f4,v4(r1)	8	11m			30
2	daddui r1,r1,8	9	11i		13	31
2	daddi r2,r2,-1	9	12i		13	31
2	bnez r2,loop	10	14j			32
3	l.d f2,v3(r1)	11	14m	15	16	32
3	l.d f3,v4(r1)	11	15m	16	17	33
3	mul.d f4,f2,f3	12	18x		24	33
3	l.d f1,v1(r1)	12	16m	17	18	34
3	div.d f4,f1,f4	13	29d		37	38
3	s.d f4,v4(r1)	13	17m			38
3	daddui r1,r1,8	14	17i		19	39
3	daddi r2,r2,-1	14	18i		19	39
3	bnez r2,loop	15	20j			40



Nome, MATRICOLA

Domanda 4

Considerando il segmento di codice presentato nella tabella precedente, se assumiamo che si vorrebbero migliorare le prestazioni del programma duplicando una delle seguenti unità funzionali:

- FP multiplier unit
- FP divider unit
- FP Arithmetic unit

Quale permetterebbe di ottenere un miglioramento maggiore nelle prestazioni del processore? motivare la risposta.

La duplicazione dell' unità di divisione permetterebbe un miglioramento delle prestazioni perché il codice attuale crea stalli strutturali che con un'ulteriore unità funzionale si potrebbero evitare.